The Wayback Machine - https://web.archive.org/web/20241206102505/https://medium.com/@ApacheDoris/r…

Open in app ↗ Sign up Sign in

Medium Search

Real-time data warehouse in TikTok based on Apache Doris

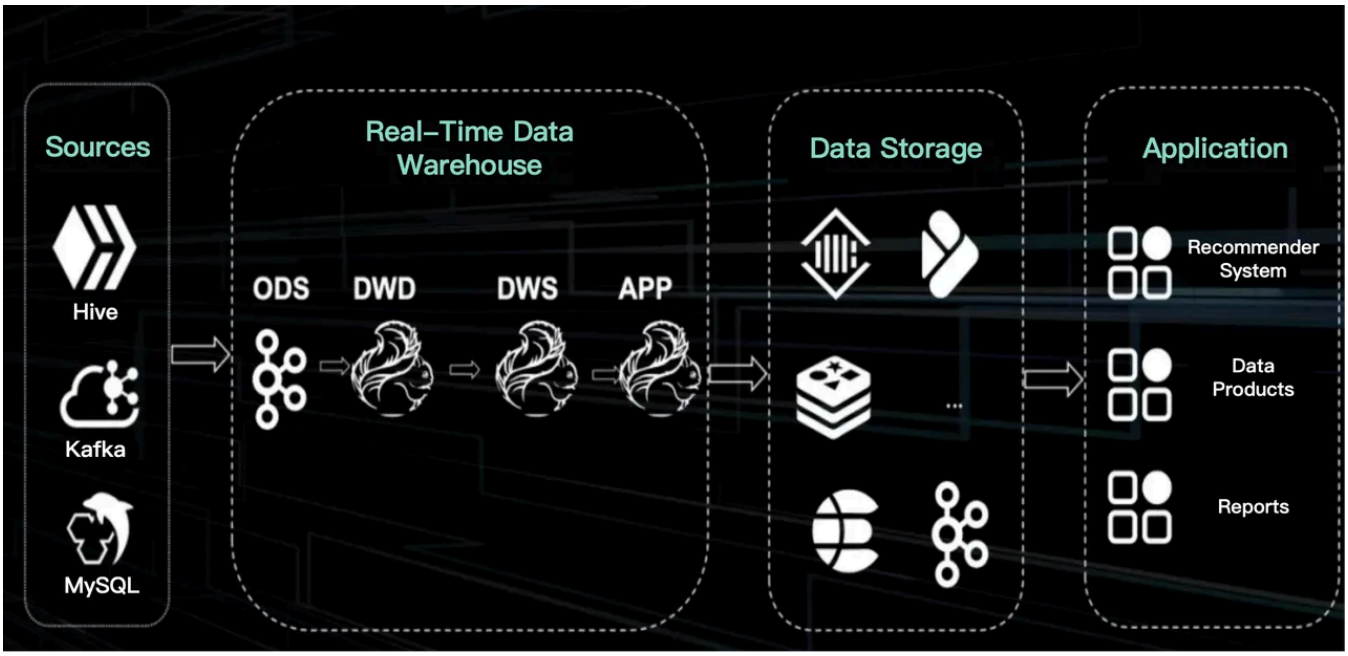
Apache Doris · Follow
5 min read · Sep 20, 2024

Listen Share

Live streaming and e-commerce are among the biggest revenue drivers of TikTok, and they both rely on real-time data processing. It is more challenging than offline batch processing because it involves complicated operations like multi-stream JOINs and dimension table changes. It requires a higher level of development and maintenance input, and due to the need for system stability guarantee, it often leads to resource redundancy and waste.

We are excited to invite the data platform team of TikTok to talk about how they use Apache Doris in their real-time data architecture and how they benefit from it, which could serve as a model for effective real-time data warehousing.

Real-time data warehouse in TikTok



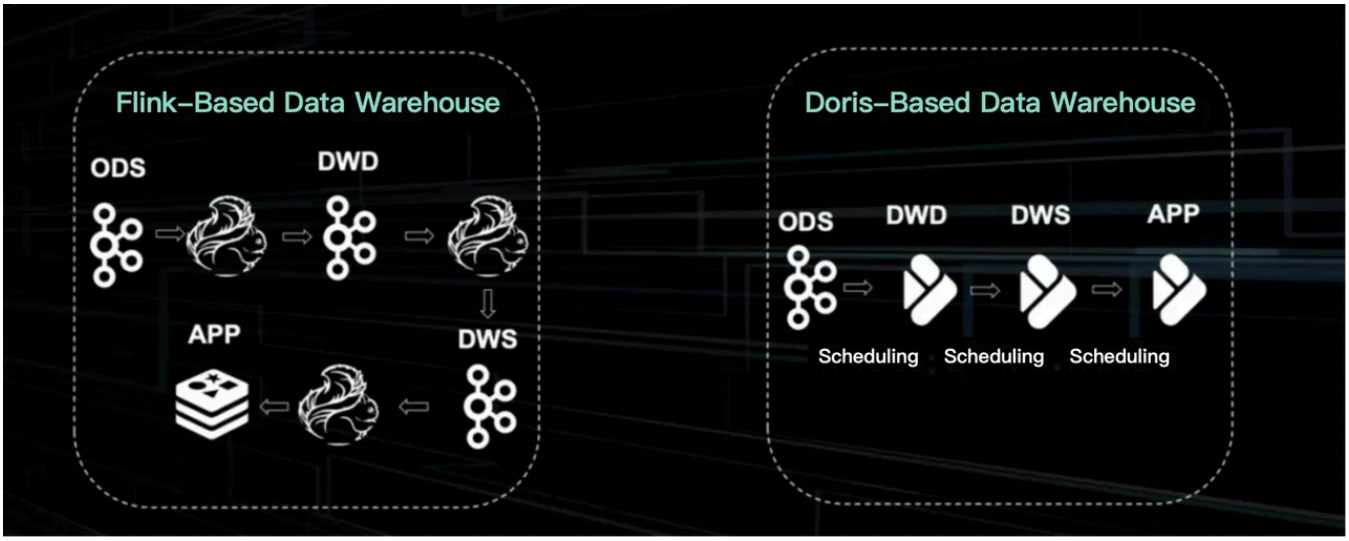
Before migration to Apache Doris, the real-time data was transferred by Flink, with Kafka enabling data flow between different data layers. Because Kafka itself does not have a logical table, it was hard to develop on it as easily as on Hive.

For TikTok, there is a significant gap in the data volume between real-time data and offline data. Due to the costs in development, operation, and resources associated with real-time data, the team tended to downgrade the real-time data requirements, but it was only a duct-tape solution.

- **Development costs**: Since Flink is a stateful data stream engine with incremental state, it requires developers to have better knowledge of the underlying architecture, especially in multi-stream JOINs. The incremental state makes it impossible to store the full data state in memory like Hive. Real-time data requires a huge storage volume and the use of various computing engines (OLTP engines like MySQL, OLAP engines like ClickHouse and Apache Doris, and KV stores like Abase, Tier, and Redis) to meet different computing needs, adding to the development complexity. The incremental state also makes testing more challenging.
- **Maintenance costs**: Complex multi-stream JOIN operations often require storing large amounts of state data, which can lead to stability issues, especially when processing continuous live streams. The live stream business of TikTok is under continuous innovation. When there is a data schema change, direct deployment might cause data recovery failure due to altered state structures.

- **Resources**: Under-utilization of resources is a common issue in real-time scenarios. For example, at the beginning of a sales campaign, there is often a tidal surge of traffic, which will quickly decrease after a few minutes. However, to ensure stability throughout the campaign, a high resource level must be maintained 24/7, leading to resource waste.

TikTok was looking for a solution that provides the same convenience as offline data development and reduces resource costs. Additionally, since the intermediate state data of Flink is inaccessible, it will be great that the new solution can enable quick and easy data testing, which is a prerequisite for data accuracy.



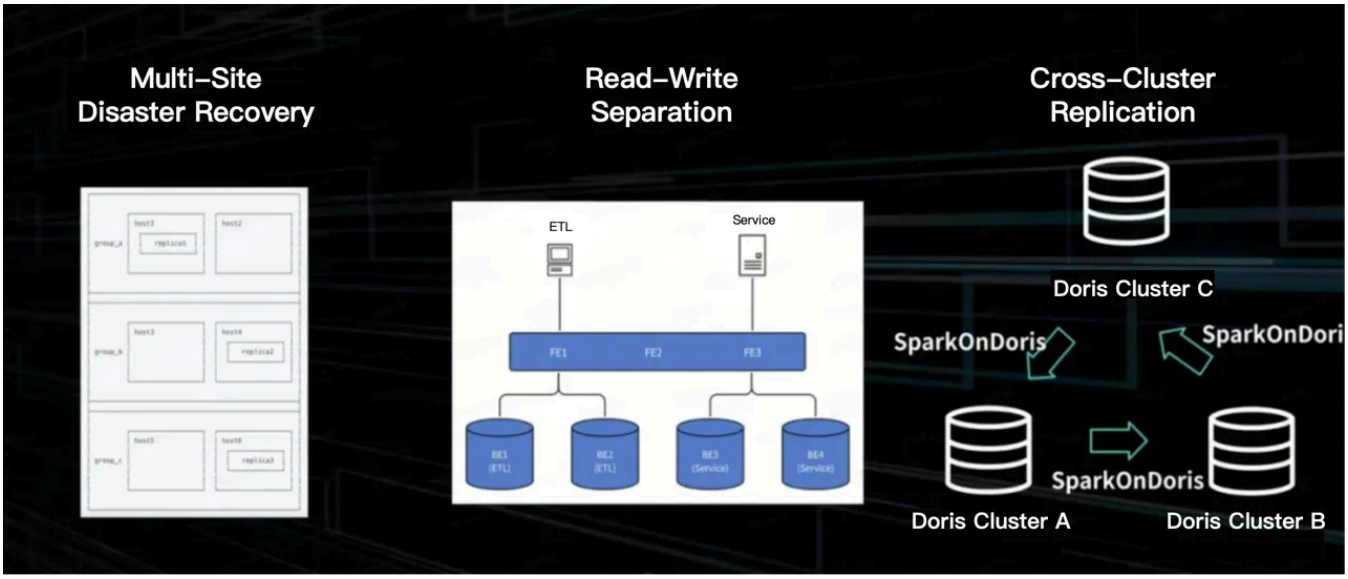
The Flink-based architecture has been a mature solution in TikTok. It was primarily used for well-established business applications. In terms of data storage, it utilized the logical table format provided by Kafka. Despite the lack of fields, constraints, and high data traceability, this logical table approach supported over half of the real-time data development.

The new architecture is based on Apache Doris. It is simpler and similar to an offline Hive setup. The key to this Doris-based architecture is the combination of the sub-second scheduling engine with the OLAP engine. This enables data layering and the reuse of offline development.

OLAP engine

To serve the live streaming business in TikTok, the OLAP engine should be well-performed in:

- **Cross-site disaster recovery**: This provides stability guarantee for live streaming to avoid significant financial losses caused by service unavailability.
- **Read-write isolation**: This is another guarantee of stability.
- **Cross-cluster ETL**: Data is scattered across different clusters for different business scenarios. For example, Cluster B and C both process transaction data, which should be synchronized from Cluster B to C, otherwise it will lead to duplicate data warehouse construction across business lines and impose burden on manpower and resources.



This is how TikTok addresses these challenges:

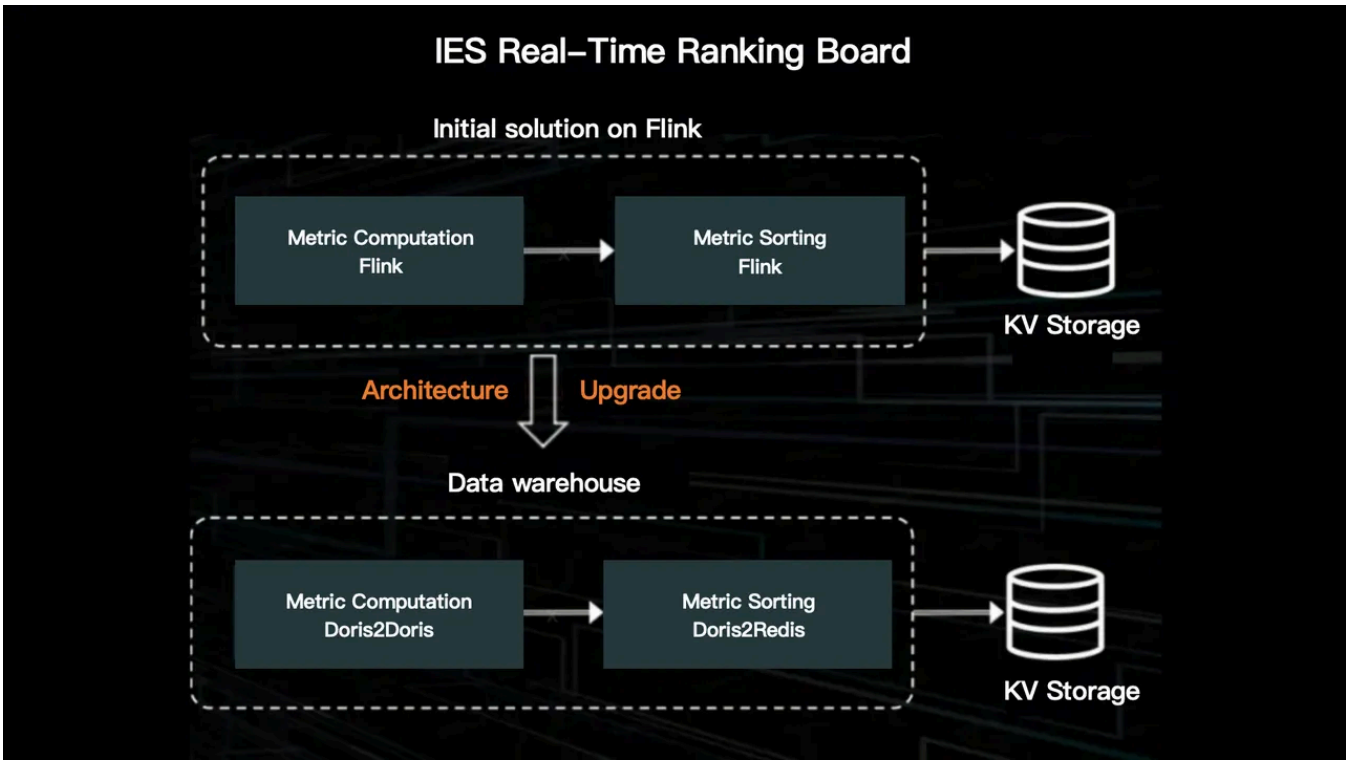
- **Cross-site disaster recovery**: Each table is stored with three replicas. The replicas are distributed across the data centers to ensure availability in each site. Message queues from the production end go through intermediate processing and arrive at the consumption end, forming the full data service link. In the event of a single data center outage, there are policies in place for both production and consumption to ensure service efficiency and stability.
- **Read-write isolation**: The read and write traffic is routed to different cluster groups.
- **Cross-cluster ETL**: For cross-cluster reads and writes, TikTok employs two mechanisms based on different business requirements and time sensitivity. One is to use Spark to read the data source format into a Yarn cluster and then synchronize it to the other clusters. The other is to leverage the cross-cluster replication capability of Apache Doris. The Spark on Doris method is more

stable and does not consume the computing resources of Doris, while the second approach is more efficient.

Real-time ranking board

How is the real-time data warehouse supporting the live streaming business of TikTok?

It builds a real-time ranking board to monitor its live streaming business performance. As is mentioned, it was migrated from Flink to Apache Doris. The new solution has a clear definition of metadata.



The metadata is parsed from the fields in the real-time tables and then given definitions. Defining the metadata is a way to abstract the business logic of the ranking board.

This also involves defining the partitioning logic of the real-time ranking board. By simple configurations, the corresponding Flink tasks can be quickly created.

However, the need for such real-time ranking boards proliferates and challenges the Flink architecture in two ways:

- Firstly, the excessive number of ranking boards causes a surge in tasks, making resource management more difficult, especially real-time stream processing that runs 24/7.

- Secondly, alerts from real-time tasks are getting more frequent. Also, the large number of tasks consuming the same message queue amplifies traffic, imposing an additional burden on HDFS.

Moreover, since large promotional events in e-commerce tend to run for a long period, the long-cycle computation is a threat to the stability of Flink. **This also makes backtracking difficult.** To address these issues, maintenance personnel often need to perform operations at midnight when the state is relatively small and backtracking pressure is reduced.

Compared to the solution on Flink, the Doris-based data warehouse consumes less resources and generates less alerts. Plus, **because the states are stored in Doris tables, long-cycle computation becomes more flexible.**

Join Apache Doris community on Slack!

Data Engineering | Data Science | Database | Big Data | Tiktok

Follow

Written by Apache Doris

1.6K Followers · 13 Following

An open-source real-time data warehouse. https://doris.apache.org https://shorturl.at/fvzVW

No responses yet